



CineFind

Sistema de Gestión de Películas por Consola

Grupo 8 · Python · 2026

¿Qué es CineFind?

CineFind es una aplicación de consola desarrollada en Python que permite gestionar un catálogo personal de películas.

El usuario puede agregar, buscar, actualizar y visualizar películas, junto con datos como género, puntaje y año de estreno.

4

Listas
paralelas

8

Opciones
de menú

1

Archivo
JSON



Estructura de Datos

[0]

películas[]

Nombres de las
películas

[1]

generos[]

Género de cada
título

[2]

puntajes[]

Calificación
0 – 10

[3]

años[]

Año de estreno
1888–2026

El índice i vincula la misma película en las 4 listas

Persistencia con JSON

cargar_datos()

- Verifica si existe el archivo con `os.path.exists()`
- Si existe → lee y desempaqueta las 4 listas del JSON
- Si no existe → inicializa 5 películas hardcodeadas por defecto

guardar_datos()

- Empaqueta las 4 listas en un diccionario Python
- Sobreescribe el JSON con `json.dump()`
- Se llama luego de agregar o actualizar una película

⚠ La opción 'Ordenar' no guarda cambios – opera sobre copias temporales



Menú Principal

1

Agregar película

2

Mostrar películas

3

Actualizar película

4

Calcular promedio

5

Ordenar por puntaje

6

Buscar película

7

Ver estadísticas

8

Películas destacadas

0

Salir (pide mail)

Validaciones de Entrada

`pedir_nombre() / pedir_genero()`

Loop while

Rechaza cadenas vacías

`pedir_puntaje()`

try / except ValueError

Acepta 0.0–10.0, rechaza letras

`pedir_año()`

`re.fullmatch(\d{4})`

Rango 1888–2026, 4 dígitos

`pedir_mail()`

`re.match(patron)`

Formato usuario@dominio.com

`existe_pelicula()`

`re.fullmatch + IGNORECASE`

Evita duplicados sin importar mayúsculas



Agregar y Buscar Películas

agregar_pelicula()

1. Solicita el nombre con pedir_nombre()
2. Llama a existe_pelicula() → aborta si ya existe
3. Pide género, puntaje y año con sus validaciones
4. Hace .append() en las 4 listas en paralelo
5. Llama a guardar_datos() para persistir

buscar_pelicula()

Llama a diccionarioPelicula() internamente

Usa re.fullmatch + IGNORECASE para encontrar la posición

Devuelve un dict con pelicula, genero, puntaje y año

⚠ Requiere el nombre exacto (no busca parcial)

⚠ Caracteres especiales de regex pueden dar errores inesperados



Ordenamiento: Bubble Sort

¿Cómo funciona?

- Recorre la lista comparando pares adyacentes de puntajes
- Si $\text{puntajes}[j] < \text{puntajes}[j+1] \rightarrow$ hace swap
- El swap se replica simultáneamente en las otras 3 listas para mantener la sincronía de datos
- Resultado: orden descendente (mayor puntaje primero)
- Opera sobre COPIAS de las listas $\rightarrow$ el catálogo original no se modifica ni se guarda

.copy()

Antes de ordenar se crea una shallow copy de cada lista.

Es seguro porque las listas contienen strings y números (tipos inmutables).

Complejidad

Peor caso: $O(n^2)$

Mejor caso: $O(n)$

N = cantidad de películas



Estadísticas y Destacadas

estadisticas(puntajes)

`max(puntajes)` → Puntaje máximo

`min(puntajes)` → Puntaje mínimo

`sum / len` → Promedio redondeado

`try / except (ZeroDivisionError, ValueError)`

Captura lista vacía en una sola línea

mostrar_destacadas()

Genera una lista de índices [0, 1, 2 ...]

Aplica `filter()` con lambda i: `puntajes[i] > 8`

Filtra ÍNDICES, no valores directamente

Usa los índices resultado para acceder a `peliculas[i]` y `puntajes[i]`

Criterio: puntaje estrictamente mayor a 8

`promedio_puntajes()` usa `reduce(lambda a,b: a+b, puntajes, 0) / len()`



Matriz Bidimensional

`matriz_peliculas(filas, columnas, peliculas)`

- Organiza los nombres en una lista de listas
- Dimensión fija: 3 filas × 5 columnas = 15 celdas
- Rellena con 'VACIO' si hay menos películas
- Se regenera cada vez que se agrega una película
- `imprimir_matriz()` recorre con doble for y usa | como separador

Ejemplo visual (3 × 5):

Matrix	Titanic	Inception	Gladiator	Interestelar
hola	agustin	VACIO	VACIO	VACIO
VACIO	VACIO	VACIO	VACIO	VACIO

⚠ Películas más allá de la posición 14 no se muestran en la matriz



Testing del Proyecto

pytest — test_cinefind.py

test_pelicula_existente()

→ True — 'Matrix' está en el catálogo

test_pelicula_no_existente()

→ False — 'Avatar' no existe

test_pelicula_ignora_mayusculas()

→ True — 'tItAnIc' == 'Titanic'

doctest — en existe_pelicula()

- Los mismos 3 casos están escritos en el docstring de la función
- Se ejecutan con `doctest.testmod()` al correr el programa directamente (`if __name__ == '__main__':`)
- Esto hace que la función sirva como documentación y como test a la vez
- Los casos cubren: existe exacto, no existe, y case-insensitive

Conclusiones



Listas paralelas

Estructura simple y efectiva para datos relacionados sin usar clases u objetos

Persistencia JSON

cargar_datos / guardar_datos dan vida al catálogo entre sesiones

Validaciones robustas

Combinación de loops, excepciones y regex cubre todos los tipos de entrada

Funciones de orden superior

reduce, filter y lambda aplicados de forma práctica y significativa

Testing dual

pytest para regresión formal + doctest para documentación ejecutable